

AgentFM: Role-Aware Failure Management for Distributed Databases with LLM-Driven Multi-Agents

Lingzhe Zhang[†]
Peking University
Beijing, China
zhang.lingzhe@stu.pku.edu.cn

Yunpeng Zhai[†]
Alibaba Group
Beijing, China
zhaiyunpeng.zyp@alibaba-inc.com

Tong Jia*
Peking University
Beijing, China
jia.tong@pku.edu.cn

Xiaosong Huang
Peking University
Beijing, China
hxs@stu.pku.edu.cn

Chiming Duan
Peking University
Beijing, China
duanchiming@stu.pku.edu.cn

Ying Li*
Peking University
Beijing, China
li.ying@pku.edu.cn

ABSTRACT

Distributed databases are critical infrastructures for today’s large-scale software systems, making effective failure management essential to ensure software availability. However, existing approaches often overlook the role distinctions within distributed databases and rely on small-scale models with limited generalization capabilities. In this paper, we conduct a preliminary empirical study to emphasize the unique significance of different roles. Building on this insight, we propose **AgentFM**, a role-aware failure management framework for distributed databases powered by LLM-driven multi-agents. AgentFM addresses failure management by considering system roles, data roles, and task roles, with a meta-agent orchestrating these components. Preliminary evaluations using Apache IoTDB demonstrate the effectiveness of AgentFM and open new directions for further research.

CCS CONCEPTS

• **Software and its engineering** → **Maintaining software.**

KEYWORDS

Failure Management, Distributed Databases, Multi Agents

ACM Reference Format:

Lingzhe Zhang[†], Yunpeng Zhai[†], Tong Jia*, Xiaosong Huang, Chiming Duan, and Ying Li*. 2025. AgentFM: Role-Aware Failure Management for Distributed Databases with LLM-Driven Multi-Agents. In *33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion ’25)*, June 23–28, 2025, Trondheim, Norway. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3696630.3728492>

[†]Equal contribution.

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

FSE Companion ’25, June 23–28, 2025, Trondheim, Norway

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1276-0/25/06...\$15.00

<https://doi.org/10.1145/3696630.3728492>

1 INTRODUCTION

The distributed databases, such as Google Spanner [2], Alibaba OceanBase [16], TiDB [5], and Apache IoTDB [14], have become integral components of cloud infrastructures, handling vast volumes of data [7, 25].

However, these systems frequently encounter anomalies such as system failures and performance degradation, leading to significant financial losses. For example, Alibaba Cloud faces Intermittent Slow Queries (ISQs) [11], leading to billions of dollars in annual losses. Amazon reports that even a 0.1-second delay in loading caused by database anomalies can lead to a 1% increase in financial losses [19]. Therefore, it is crucial to detect system failures in real time, analyze the root causes of these failures, and automatically remediate them.

System traces, metrics, and logs capture the states and critical events of active processes, making them essential for managing software failures. These data sources provide insights into both normal operations and deviations signaling potential failures. Leveraging their multimodal nature, recent research has enhanced anomaly detection and diagnosis in complex systems [6, 8–10, 17, 20–24, 26, 29, 30]. For example, Eadro [8] integrates anomaly detection and root cause localization using multi-source data, while AnoFusion [26] employs unsupervised multimodal failure detection in microservices. MSTGAD [6] combines all three data types with attentive multimodal learning for graph-based anomaly detection. Studies also explore two-source combinations, such as DeepTraLog [17], which pairs traces with logs using a GGNN-based model, and SCWarn [29], which identifies anomalies via heterogeneous metrics and logs. Although the effectiveness of these methods has been demonstrated in specific scenarios, they face several practical challenges when applied to distributed databases:

- **Role Awareness.** In distributed databases, nodes play different roles, each with varying levels of importance. Effective failure management often requires coordination and collaboration among multiple nodes. However, current methods completely overlook these roles, leading to suboptimal fault detection, inaccurate root cause diagnosis, and an inability to provide critical repair strategies.
- **Application Limitations.** Existing failure management models lack generalizability. For instance, these models are typically trained on specific systems and encounter concept drift when applied to new systems. Moreover, since these

methods often frame the task as a classification problem, the resulting outputs lack interpretability, which is crucial for Operations and Control Engineers (OCEs) to effectively resolve issues.

To address the first challenge, we conduct a detailed analysis of the various roles within a distributed database and the entire failure management process. We ultimately identify three distinct role categories: system roles, data roles, and task roles. System roles represent the various roles inherent in the distributed database itself (e.g., leader nodes, follower nodes). Data roles refer to the different data sources involved in failure management, while task roles represent the various tasks that need to be executed during failure management.

To tackle the second challenge, we adopt a large language model (LLM)-based approach. While many existing LLM-based failure management solutions have been proposed, a significant number of them do not utilize the three types of data sources mentioned above [1, 3, 12, 18, 28]. Moreover, some approaches that do incorporate these data sources fail to account for the role-based structure within distributed databases [4, 13, 27]. Therefore, we propose a role-aware, LLM-driven multi-agent approach that integrates the characteristics of these roles and the three data sources.

Building on these insights, we introduce AgentFM, a comprehensive role-aware failure management framework for distributed databases, powered by LLM-driven multi-agent systems. AgentFM integrates the unique characteristics of distributed database roles with the rich multimodal data sources typically encountered in failure management, such as system traces, metrics, and logs. By employing a multi-agent architecture, AgentFM facilitates specialized agents for each role—system roles, data roles, and task roles—ensuring a more nuanced and effective approach to failure detection, diagnosis, and resolution.

We conduct preliminary experiments on Apache IoTDB [14], a distributed time-series database system, to assess the effectiveness of AgentFM in failure detection and root cause analysis. Furthermore, we manually verified the accuracy and validity of the generated mitigation solution.

Contributions. The contributions of this paper are threefold. First, we conduct a preliminary empirical study highlighting the varying significance of different roles in failure management. Second, we propose AgentFM, a role-aware failure management framework for distributed databases with LLM-driven multi-agents. Third, we provide a preliminary evaluation of AgentFM on Apache IoTDB, demonstrating its feasibility and effectiveness.

2 PRELIMINARY EMPIRICAL STUDY

In this section, we conduct a preliminary empirical study using Apache IoTDB [14], focusing on the significance of different roles in the failure management process.

2.1 System Roles

We manually injected anomalies (excessive data export) into Apache IoTDB during runtime and evaluate the anomaly detection performance (precision, recall and f1-score) for each node using PLELog [15], which is a state-of-the-art log-based anomaly detection method.

Table 1: Anomaly Detection Results for Each Node

	<i>Node₁</i>	<i>Node₂</i>	<i>Node₃</i>	<i>Node₄</i>	<i>Node₅</i>	<i>Node₆</i>
p	39.68%	34.35%	59.64%	31.17%	77.60%	89.42%
r	99.01%	100.00%	98.02%	100.00%	96.04%	92.08%
f1	56.66%	51.14%	74.16%	47.53%	85.84%	90.73%

As shown in Table 1, *Node₆* achieves the best anomaly detection performance due to hosting the largest number of leader partitions. In contrast, the detection performance on other nodes is relatively suboptimal. This observation underscores the differing significance of various system roles.

2.2 Data Roles

We further conduct anomaly diagnosis classification experiments on Apache IoTDB using both metrics data and log data. To ensure fairness, we do not adopt state-of-the-art methods; instead, we implement a simple classification algorithm based on the Dynamic Time Warping (DTW) algorithm.

Table 2: Anomalies that can be Classified by Metrics and Logs

Source	CPU	Memory	Export	Import	Configuration
Metrics	✓	✓			
Logs			✓	✓	✓

As shown in Table 2, anomalies with an F1-score above 50% are considered identifiable by the corresponding data type. The results reveal that metrics are particularly effective at detecting resource anomalies, such as CPU and memory saturation, while logs excel at identifying internal database issues, including excessive data export/import and configuration errors. This observation highlights the distinct significance of different data roles.

3 METHODOLOGY

Our preliminary empirical study highlights that the effectiveness of failure management varies significantly across different roles in distributed databases. Consequently, it becomes crucial to assign varying levels of importance to different roles at different times and adopt role-specific operations to optimize outcomes.

In this section, we introduce **AgentFM**, a role-aware failure management framework for distributed databases powered by LLM-driven multi-agents. Figure 1 illustrates the architecture of AgentFM, which comprises three types of agents corresponding to distinct roles: system agents, data agents, and task agents. System agents represent the various roles of nodes within the distributed database, data agents handle information from diverse data sources, and task agents focus on executing specific failure management tasks. Notably, since system agents and data agents operate independently on each node, they are collectively referred to as standalone agents. Additionally, the framework incorporates a meta-agent, which is responsible for orchestrating, adapting, aggregating, and ultimately producing the final results of the agents' operations, ensuring cohesive and efficient failure management.

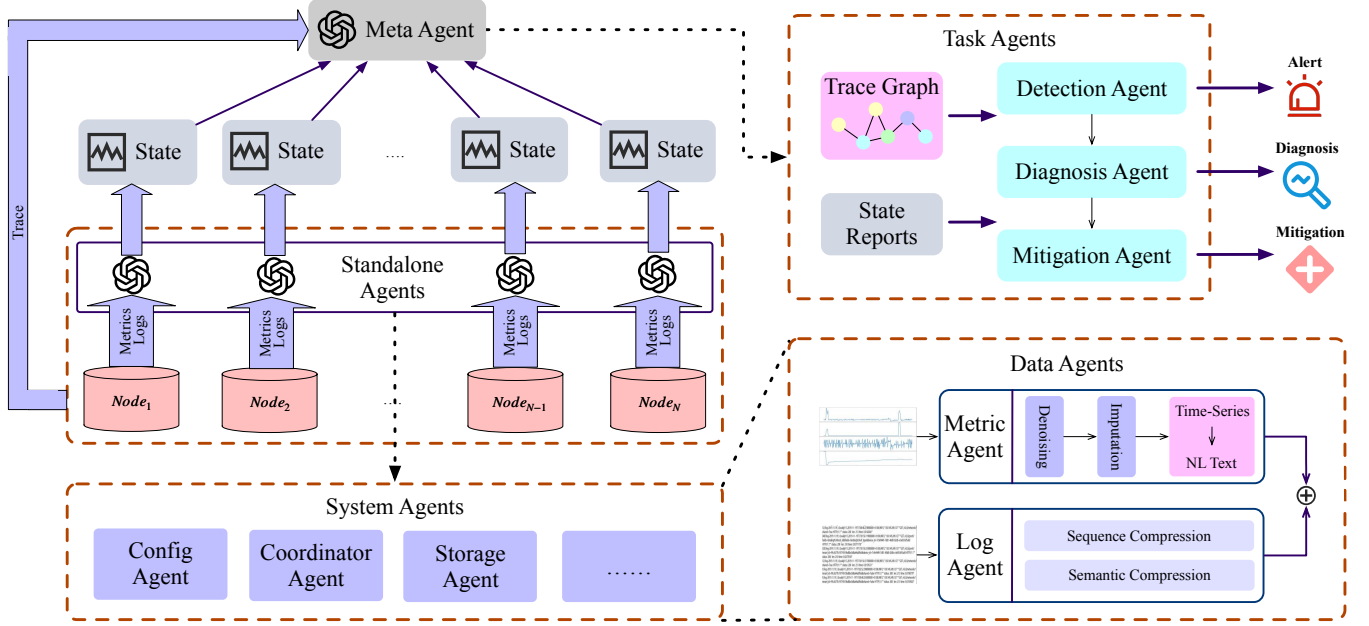


Figure 1: AgentFM Architecture

The framework operates by first extracting critical information from different system agents using the corresponding data agents. This information is then transmitted to a centralized meta-agent for unified decision-making. The decision-making process employs trace data as a cohesive framework to integrate key insights across nodes. Finally, the appropriate task agents execute the required failure management operations, completing the process efficiently and effectively.

3.1 System Agents

Different distributed databases assign dynamic system roles that may change during runtime (e.g., leader re-election after node failure). To handle this, we designed an adaptive mechanism for system agents within the Meta Agent.

As illustrated in Figure 2, the core component is the System Role Manager, which initializes by analyzing configuration files and system docs to identify each node's role and importance. At runtime, it periodically queries the system state and updates roles accordingly.

Based on this process, each node instantiates a corresponding system agent—such as a Config Agent, Coordinator Agent, or Storage Agent—each embedding multiple data agents to handle specialized tasks.

3.2 Data Agents

In this paper, we define two types of data agents: the Metric Agent and the Log Agent. Each adopts a distinct approach to extract key information from the raw metrics data and log data, respectively.

Metric Agent: The overall workflow of the Metric Agent begins with simple data preprocessing, which includes noise removal and imputation of missing values. After preprocessing, the multivariate

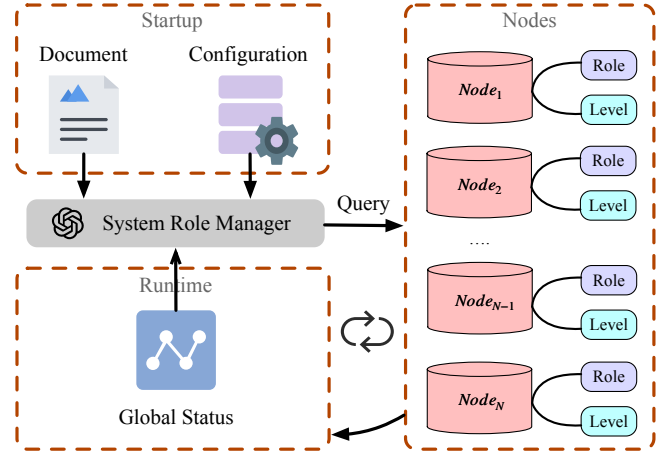


Figure 2: System Agents Adaptation Workflow

time-series data is converted into natural language descriptions using a large model. The generated natural language descriptions capture key aspects of the data, including time intervals, fluctuation trends, and anomaly points.

Formally, let $\mathbf{M} = \{m_1, m_2, \dots, m_n\}$ represent the raw multi-dimensional metrics data, where each m_i corresponds to a time-series for a specific metric. The preprocessing step can be represented as Equation 1, where $\mathbf{M}_p$ is the denoised and imputed version of $\mathbf{M}$.

$$\mathbf{M}_p = \text{Preprocess}(\mathbf{M}) = \{m'_1, m'_2, \dots, m'_n\} \quad (1)$$

Next, the processed data is converted into natural language descriptions using a large language model $\mathcal{L}$, as shown in Equation 2,

where D_{nl} represents the resulting natural language description, which includes information on time intervals, trends, and identified anomalies in the data.

$$D_{nl} = \mathcal{L}(M_p) \quad (2)$$

Log Agent: Unlike metrics data, log data is written by developers to monitor the internal state of the system, inherently containing rich semantic information. However, log data often includes substantial redundant information, such as repetitive log entries and messages unrelated to system anomalies.

To address this, the Log Agent comprises two main components: sequence compression and semantic compression. Sequence compression focuses on reducing the raw log sequence by consolidating repetitive log patterns, while semantic compression extracts key operational information from the logs.

Sequence Compression: This component utilizes a log-parsing algorithm to transform each log entry into a distinct event template consisting of a static constant part and variable parameters. It then merges consecutive occurrences of identical event templates. Formally, for a given raw log sequence $L = \{l_1, l_2, \dots, l_N\}$, where l_n represents an individual log entry, the parsing process transforms L into $L_e = \{e_1, e_2, \dots, e_N\}$, where e_i represents an event template. Consecutive identical events are merged, yielding $G' = \{e'_1, e'_2, \dots, e'_C\}$, where $C \ll N$ and $e'_i = e_i \times c$ represents the event e_i with a count c .

Semantic Compression: This component employs a prompt-based method to convert lengthy log sequences into descriptions of the database's ongoing operations. Formally, given a sequence of raw log entries $L = \{l_1, l_2, \dots, l_N\}$, the LLM-based summarization transforms and compresses L into a smaller set of key operational elements $O = \{o_1, o_2, \dots, o_M\}$, where $M \ll N$.

3.3 Task Agents

After system agents collect necessary data via data agents, the meta agent coordinates task agents to carry out failure management using trace data. This process involves three agent types—detection, diagnosis, and mitigation—which operate sequentially. Detection agents identify anomalies in each time window; upon detection, diagnosis agents locate and classify the issue; finally, mitigation agents propose solutions based on the diagnosis.

Though prompts vary by task, all agents follow a common RAG+CoT approach, using historical data as labeled examples—normal/abnormal for detection, failure types for diagnosis—to guide reasoning during querying.

4 PRELIMINARY EVALUATION

4.1 Design

To evaluate AgentFM, we assess its feasibility and effectiveness in Apache IoTDB. We manually injected 10 types of anomalies, including CPU saturation, IO saturation, memory saturation, network delay increase, network bandwidth limitation, network partition occurrence, workload spikes, accompanying slow queries, excessive data export, and excessive data import. Each anomaly type is injected 20 times.

The evaluation is conducted based on Qwen2.5-72b to assess the results of anomaly detection and diagnosis. The performance is

measured using precision, recall, and F1-score metrics. Additionally, the mitigation solutions generated are manually analyzed to determine their usefulness.

4.2 Results

We utilize the function call feature to guide the model in generating "yes" or "no" responses to evaluate anomaly detection performance. For failure diagnosis, we manually review the LLM-generated results to identify the classification most closely matching the pre-injected anomalies and assessed the outcomes accordingly.

Table 3: Anomaly Detection & Diagnosis Results

Task	Precision	Recall	F1-Score
Anomaly Detection	95.14%	97.03%	95.76%
Failure Diagnosis	89.61%	87.04%	87.62%

As shown in Table 3, AgentFM demonstrates promising performance in our preliminary experiments. However, based on our observations, the model struggles to identify correct results when dealing with complex internal anomalies (e.g., excessive data import). We plan to focus on optimizing performance for such scenarios in future work.

Increase CPU Resources:

Iotdb0 frequently experiences high CPU usage, consider increasing the number of CPU cores allocated to the container or raising the CPU limit.

Load Balancing:

If iotdb0 is part of a cluster, consider distributing some of the load to other nodes to achieve load balancing.

Figure 3: Sample Mitigation Solutions from AgentFM

We also conduct a manual analysis of the mitigation solutions recommended by AgentFM. One example is illustrated in Figure 3. We extract the key information from the solution and find that these recommendations provide targeted and effective assistance in system remediation.

5 CONCLUSION

This paper investigates the varying significance of different roles in failure management. Building on this insight, we propose AgentFM, a role-aware failure management framework for distributed databases powered by LLM-driven multi-agents. Our preliminary experiments demonstrate the feasibility of AgentFM. In the future, we plan to further enhance AgentFM's capabilities in automating remediation execution.

ACKNOWLEDGEMENT

This work is supported by Key-Area Research and Development Program of Guangdong Province, China (NO.2020B010164003).

REFERENCES

- [1] Toufique Ahmed, Supriyo Ghosh, Chetan Bansal, Thomas Zimmermann, Xuchao Zhang, and Saravan Rajmohan. 2023. Recommending root-cause and mitigation steps for cloud incidents using large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1737–1749.
- [2] James C Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, Jeffrey John Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, et al. 2013. Spanner: Google’s globally distributed database. *ACM Transactions on Computer Systems (TOCS)* 31, 3 (2013), 1–22.
- [3] Drishti Goel, Fiza Husain, Aditya Singh, Supriyo Ghosh, Anjaly Parayil, Chetan Bansal, Xuchao Zhang, and Saravan Rajmohan. 2024. X-lifecycle learning for cloud incident management using llms. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 417–428.
- [4] Adha Hrusto, Per Runeson, and Magnus C Ohlsson. 2024. Autonomous monitors for detecting failures early and reporting interpretable alerts in cloud operations. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*. 47–57.
- [5] Dongxu Huang, Qi Liu, Qiu Cui, Zhuhe Fang, Xiaoyu Ma, Fei Xu, Li Shen, Liu Tang, Yuxing Zhou, Menglong Huang, et al. 2020. TiDB: a Raft-based HTAP database. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3072–3084.
- [6] Jun Huang, Yang Yang, Hang Yu, Jianguo Li, and Xiao Zheng. 2023. Twin graph-based anomaly detection via attentive multi-modal learning for microservice system. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 66–78.
- [7] Yuyuan Kang, Xiangdong Huang, Shaoxu Song, Lingzhe Zhang, Jialin Qiao, Chen Wang, Jianmin Wang, and Julian Feinauer. 2022. Separation or not: On handling out-of-order time-series data in leveled lsm-tree. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 3340–3352.
- [8] Cheryl Lee, Tianyi Yang, Zhuangbin Chen, Yuxin Su, and Michael R Lyu. 2023. Eadro: An end-to-end troubleshooting framework for microservices on multi-source data. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1750–1762.
- [9] Cheryl Lee, Tianyi Yang, Zhuangbin Chen, Yuxin Su, Yongqiang Yang, and Michael R Lyu. 2023. Heterogeneous anomaly detection for software systems via semi-supervised cross-modal attention. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1724–1736.
- [10] Cheng-Ming Lin, Ching Chang, Wei-Yao Wang, Kuang-Da Wang, and Wen-Chih Peng. 2024. Root Cause Analysis in Microservice Using Neural Granger Causal Discovery. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 206–213.
- [11] Minghua Ma, Zheng Yin, Shenglin Zhang, Sheng Wang, Christopher Zheng, Xinhao Jiang, Hanwen Hu, Cheng Luo, Yilin Li, Nengjun Qiu, et al. 2020. Diagnosing root causes of intermittent slow queries in cloud databases. *Proceedings of the VLDB Endowment* 13, 8 (2020), 1176–1189.
- [12] Devjeet Roy, Xuchao Zhang, Rashi Bhavne, Chetan Bansal, Pedro Las-Casas, Rodrigo Fonseca, and Saravan Rajmohan. 2024. Exploring llm-based agents for root cause analysis. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 208–219.
- [13] Manish Shetty, Yinfang Chen, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Xuchao Zhang, Jonathan Mace, Dax Vandevoorde, Pedro Las-Casas, Shachee Mishra Gupta, et al. 2024. Building AI Agents for Autonomous Clouds: Challenges and Design Principles. In *Proceedings of the 2024 ACM Symposium on Cloud Computing*. 99–110.
- [14] Chen Wang, Xiangdong Huang, Jialin Qiao, Tian Jiang, Lei Rui, Jinrui Zhang, Rong Kang, Julian Feinauer, Kevin A McGrail, Peng Wang, et al. 2020. Apache iotdb: time-series database for internet of things. *Proceedings of the VLDB Endowment* 13, 12 (2020), 2901–2904.
- [15] Lin Yang, Junjie Chen, Zan Wang, Weijing Wang, Jiajun Jiang, Xuyuan Dong, and Wenbin Zhang. 2021. Semi-supervised log-based anomaly detection via probabilistic label estimation. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1448–1460.
- [16] Zhenkun Yang, Chuanhui Yang, Fusheng Han, Mingqiang Zhuang, Bing Yang, Zhifeng Yang, Xiaojun Cheng, Yuzhong Zhao, Wenhui Shi, Huafeng Xi, et al. 2022. OceanBase: a 707 million tpmC distributed relational database system. *Proceedings of the VLDB Endowment* 15, 12 (2022), 3385–3397.
- [17] Chenxi Zhang, Xin Peng, Chaofeng Sha, Ke Zhang, Zhenqing Fu, Xiya Wu, Qingwei Lin, and Dongmei Zhang. 2022. Deeptralog: Trace-log combined microservice anomaly detection through graph-based deep learning. In *Proceedings of the 44th international conference on software engineering*. 623–634.
- [18] Dylan Zhang, Xuchao Zhang, Chetan Bansal, Pedro Las-Casas, Rodrigo Fonseca, and Saravan Rajmohan. 2024. LM-PACE: Confidence estimation by large language models for effective root causing of cloud incidents. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 388–398.
- [19] Lingzhe Zhang, Tong Jia, Mengxi Jia, Ying Li, Yong Yang, and Zhonghai Wu. 2024. Multivariate Log-based Anomaly Detection for Distributed Database. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4256–4267.
- [20] Lingzhe Zhang, Tong Jia, Mengxi Jia, Hongyi Liu, Yong Yang, Zhonghai Wu, and Ying Li. 2024. Towards Close-To-Zero Runtime Collection Overhead: Raft-Based Anomaly Diagnosis on System Faults for Distributed Storage System. *IEEE Transactions on Services Computing* (2024).
- [21] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S Yu, and Ying Li. 2024. A Survey of AIOps for Failure Management in the Era of Large Language Models. *arXiv preprint arXiv:2406.11213* (2024).
- [22] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Hongyi Liu, and Ying Li. 2025. ScalaLog: Scalable Log-Based Failure Diagnosis Using LLM. In *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 1–5.
- [23] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Hongyi Liu, and Ying Li. 2025. XRAGLog: A Resource-Efficient and Context-Aware Log-Based Anomaly Detection Method Using Retrieval-Augmented Generation. In *AAAI 2025 Workshop on Preventing and Detecting LLM Misinformation (PDLIM)*.
- [24] Lingzhe Zhang, Tong Jia, Kangjin Wang, Mengxi Jia, Yong Yang, and Ying Li. 2024. Reducing Events to Augment Log-based Anomaly Detection Models: An Empirical Study. In *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. 538–548.
- [25] Ling-Zhe Zhang, Xiang-Dong Huang, Yan-Kai Wang, Jia-Lin Qiao, Shao-Xu Song, and Jian-Min Wang. 2024. Time-tired compaction: An elastic compaction scheme for LSM-tree based time-series database. *Advanced Engineering Informatics* 59 (2024), 102224.
- [26] Shenglin Zhang, Pengxiang Jin, Zihan Lin, Yongqian Sun, Bicheng Zhang, Sibao Xia, Zhengdan Li, Zhenyu Zhong, Minghua Ma, Wa Jin, et al. 2023. Robust failure diagnosis of microservice system through multimodal data. *IEEE Transactions on Services Computing* 16, 6 (2023), 3851–3864.
- [27] Wei Zhang, Hongcheng Guo, Jian Yang, Yi Zhang, Chaoran Yan, Zhoujin Tian, Hangyuan Ji, Zhoujun Li, Tongliang Li, Tieqiao Zheng, et al. 2024. mABC: multi-Agent Blockchain-Inspired Collaboration for root cause analysis in micro-services architecture. *arXiv preprint arXiv:2404.12135* (2024).
- [28] Xuchao Zhang, Supriyo Ghosh, Chetan Bansal, Rujia Wang, Minghua Ma, Yu Kang, and Saravan Rajmohan. 2024. Automated root causing of cloud incidents using in-context learning with gpt-4. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 266–277.
- [29] Nengwen Zhao, Junjie Chen, Zhaoyang Yu, Honglin Wang, Jiesong Li, Bin Qiu, Hongyu Xu, Wenchu Zhang, Kaixin Sui, and Dan Pei. 2021. Identifying bad software changes via multimodal anomaly detection for online service systems. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 527–539.
- [30] Lecheng Zheng, Zhengzhang Chen, Jingrui He, and Haifeng Chen. 2024. Multimodal Causal Structure Learning and Root Cause Analysis. *arXiv preprint arXiv:2402.02357* (2024).